FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO

DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTEFATOS DO PROJETO DE SOFTWARE

Sistema para Identificação da Pinta Preta na Tangerina Ponkan, orientado por Visão Computacional

Arthur Parra da Silva
Danieli Fiel Reis
Guilherme Shimada Pereira
Gustavo Kletelinger
Matheus Bertholdo de Oliveira

Registro – SP
2026

Conteúdo

1 CANVAS: Modelo de Negócio e Estratégia **3**

2 SWOT: Análise Estratégica **3**
2.1 Recomendações Estratégicas 4

3 UX/UI: Design de Interface e Experiência do Usuário **4**
3.1 Guia de Estilos 6

4 Website: Estrutura e Especificações Técnicas Web **6**
4.1 Mapa do Site da Equipe 6
4.2 Conteúdo das Páginas 7
4.3 Especificações Técnicas do Site da Equipe 8

5 Diagramas UML: Modelagem Visual do Sistema **8**
5.1 Diagrama de Casos de Uso 9
5.2 Diagrama de Classes 10
5.3 Diagrama de Objetos 11
5.4 Outros Diagramas UML Importantes 11

6 DevOps: Infraestrutura, Automação e Entrega Contínua **11**
6.1 Pipeline CI/CD 11
6.2 Configuração CI/CD 12
6.3 Infraestrutura como Código 13
6.4 Infraestrutura do Sistema 13
6.4.1 Arquitetura de Rede 13
6.4.2 Diagrama de Infraestrutura 13
6.5 Arquitetura de Deploy 14
6.6 Métricas e SLA 15

7 Modelagens do Banco de Dados **15**
7.1 Modelagem conceitual 15
7.2 Modelagem lógica 16

Referências **18**

Nota Importante sobre os Artefatos

Este documento apresenta exemplos de como inserir os principais artefatos de um projeto de software. Todos os artefatos podem ser inseridos de duas formas:

1. **Como imagens:** Criar os artefatos em ferramentas externas (Canva, Miro, Draw.io, Figma, etc.) e inserir como imagens PNG/JPG usando o comando `\includegraphics`
2. **Como código LaTeX:** Usar tabelas e diagramas TikZ diretamente no documento (exemplos fornecidos)

Lista de Artefatos e suas Finalidades

- **CANVAS:** Modelo de negócio e estratégia
- **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
- **UX/UI:** Design de interface e experiência do usuário
- **Website:** Estrutura e especificações técnicas web
- **Diagramas UML:** Modelagem visual do sistema
- **DevOps:** Infraestrutura, automação e práticas de entrega contínua

---

1 CANVAS: Modelo de Negócio e Estratégia

O Business Model Canvas (A. Santos et al., 2024) é uma ferramenta visual estratégica[1] que permite descrever, analisar e projetar modelos de negócio de forma clara e objetiva. O canvas é dividido em nove blocos fundamentais que cobrem as principais áreas de um negócio.



Figura 1: Business Model Canvas do Projeto

2 SWOT: Análise Estratégica

A Análise SWOT (Strengths, Weaknesses, Opportunities, Threats) ou FOFA (Forças, Oportunidades, Fraquezas e Ameaças) (**??**) é uma ferramenta fundamental para planejamento estratégico[2] que permite identificar fatores internos e externos que impactam o projeto. Analisa forças, fraquezas, oportunidades e ameaças do projeto.

[1] O Canvas foi desenvolvido por Alexander Osterwalder e é amplamente utilizado em startups e empresas estabelecidas para modelagem de negócios.

[2] A análise SWOT foi criada na década de 1960 por Albert Humphrey no Stanford Research Institute.



Figura 2: Business Model Canvas do Projeto

2.1 Recomendações Estratégicas

Com base na análise SWOT, as principais estratégias devem focar em:

- **Forças + Oportunidades:** Investir no marketing digital para popularizar o aplicativo
- **Forças + Ameaças:** Alinhar design com capacidade tecnológica de pequenos produtores
- **Fraquezas + Oportunidades:** Buscar parcerias com cooperativas e associações
- **Fraquezas + Ameaças:** Criar planos de contingência

3 UX/UI: Design de Interface e Experiência do Usuário

UX (User Experience) e UI (User Interface) (**??**) são disciplinas complementares focadas em criar produtos digitais que sejam funcionais, acessíveis e agradáveis de usar.[3] O UX concentra-se na experiência geral do usuário, enquanto o UI cuida dos elementos visuais e interativos.

[3]Ferramentas populares para design UX/UI incluem Figma, Adobe XD, Sketch e InVision.

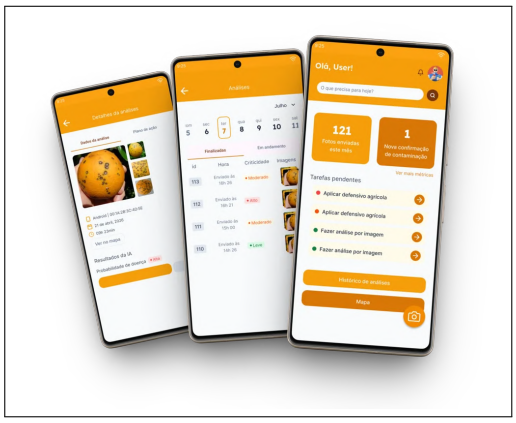


Figura 3: Mockup de alta fidelidade

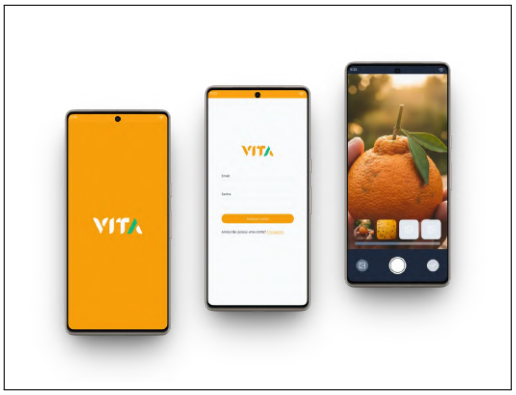


Figura 4: Mockup de alta fidelidade

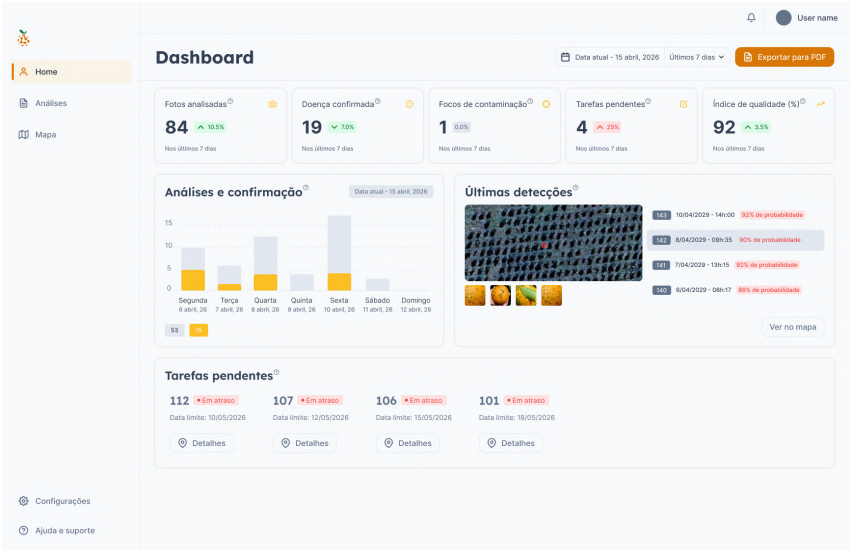


Figura 5: Protótipo da Dashboard Principal

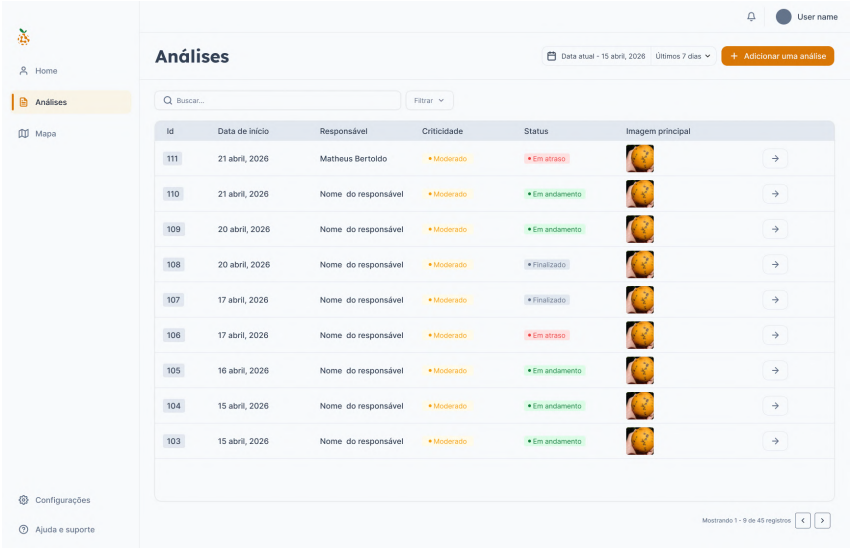


Figura 6: Protótipo do histórico de Análises

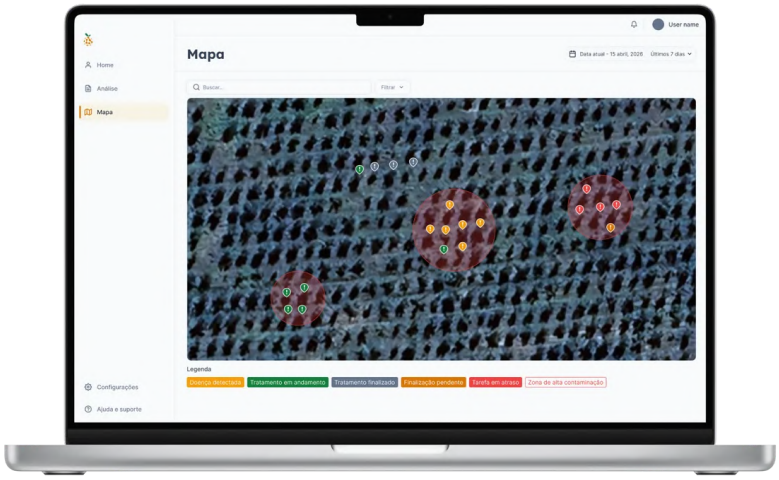


Figura 7: Mockup da tela do Mapa de Infecções

3.1 Guia de Estilos

É importante documentar o design system utilizado:

Tabela 1: Paleta de Cores do Projeto

Elemento	Cor	Código HEX	Uso
Primária		#FCA211	Botões principais, destaques
Secundária		#69AD00	Títulos, elementos secundários
Fundo		#FFFFFF	Background padrão
Texto		#333333	Texto principal
Sucesso		#7bf1a8	Mensagens de sucesso
Erro		#ff637e	Mensagens de erro

4 Website: Estrutura e Especificações Técnicas Web

Esta seção documenta o website institucional/portfólio da equipe que desenvolveu a aplicação. O site da equipe serve para apresentar os membros, projetos realizados, competências e informações de contato.

4.1 Mapa do Site da Equipe

O mapa do site (sitemap) mostra a hierarquia e organização das páginas do website da equipe:

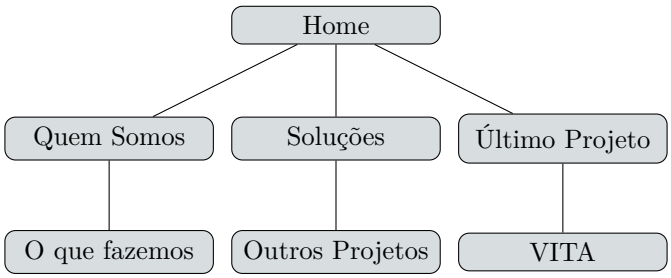


Figura 8: Estrutura de navegação do website da equipe

4.2 Conteúdo das Páginas

Páginas principais do site da equipe:

- **Quem somos:** Visão e missão da empresa
- **Soluções:** Showcase dos projetos desenvolvidos com descrições e imagens
- **Último Projeto:** Área dedicada ao projeto VITA
- **Rodapé:** Contatos, e-mail, redes sociais



Figura 9: Hero section do website



Figura 10: Carrossel de produtos do website da equipe

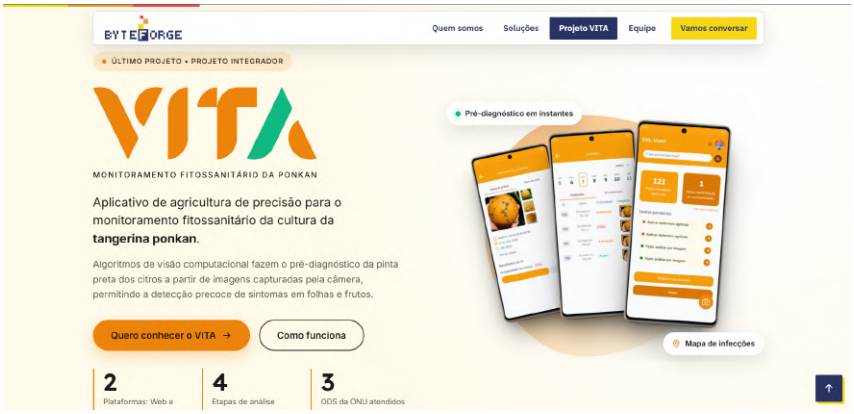


Figura 11: Apresentação do projeto VITA do website da equipe



Figura 12: Cards explicando como funciona o VITA

4.3 Especificações Técnicas do Site da Equipe

Tabela 2: Stack Tecnológica da Landing Page da Equipe

Camada	Tecnologia	Descrição
Estrutura	HTML5	Marcação semântica e acessível (WAI-ARIA)
Estilo	CSS3	Estilos modulares com variáveis para as identidades visuais
Interação	JavaScript	Scripts nativos organizados em módulos, sem frameworks
Assets	WebP/SVG/WOFF2	Mockups, logotipos e fontes otimizados para a web
Hospedagem	GitHub Pages	Hospedagem estática gratuita

5 Diagramas UML: Modelagem Visual do Sistema

A Unified Modeling Language (UML) (**??**) é uma linguagem de modelagem visual usada para especificar, visualizar, construir e documentar artefatos de sistemas de software.[4]

[4] A UML foi padronizada pela Object Management Group (OMG) em 1997.

5.1 Diagrama de Casos de Uso

O diagrama de casos de uso representa as funcionalidades do sistema do ponto de vista do usuário:



Figura 13: Diagrama de Caso de Uso do Sistema

5.2 Diagrama de Classes

O diagrama de classes mostra a estrutura estática do sistema:

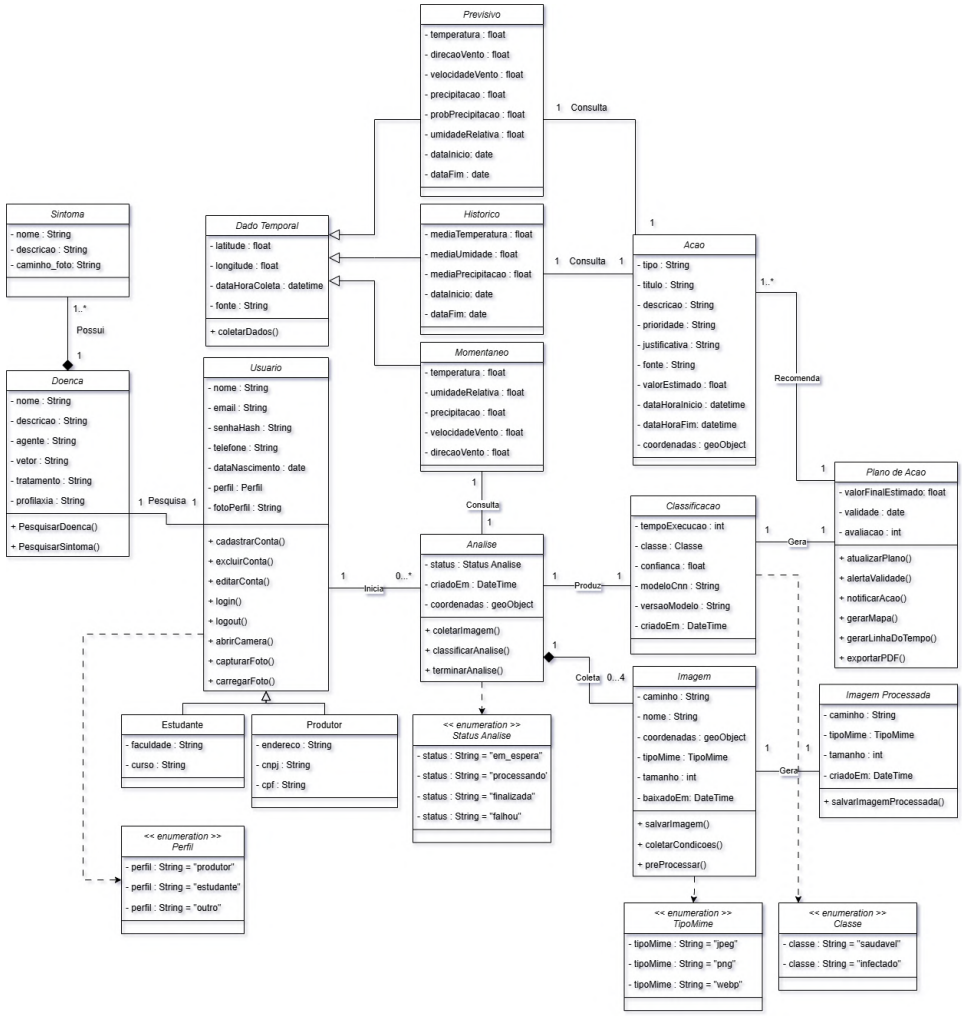


Figura 14: Diagrama de Classes do Sistema

5.3 Diagrama de Objetos

O diagrama de objetos mostra as instâncias das classes e seus dados:

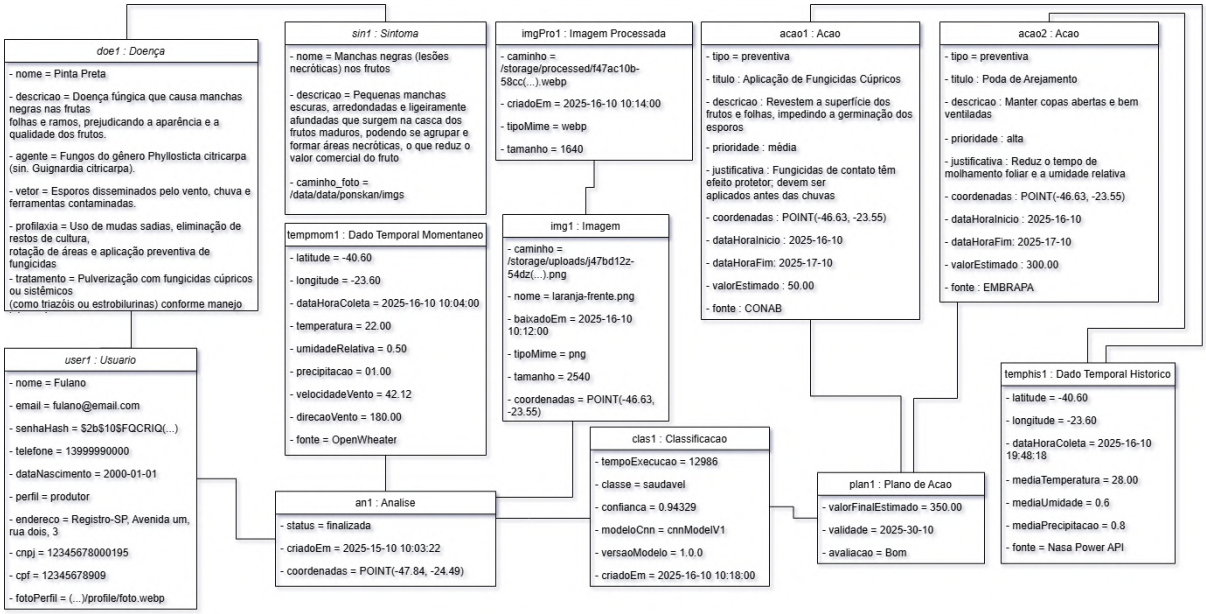


Figura 15: Diagrama de Objetos do Sistema

5.4 Outros Diagramas UML Importantes

- **Diagrama de Sequência:** Mostra a interação entre objetos ao longo do tempo
- **Diagrama de Atividades:** Representa o fluxo de trabalho ou processo
- **Diagrama de Componentes:** Mostra a organização física dos componentes
- **Diagrama de Implantação:** Representa a arquitetura física do sistema
- **Diagrama de Estados:** Mostra os estados de um objeto e suas transições

Ferramentas recomendadas: Draw.io, Lucidchart, PlantUML, StarUML, Enterprise Architect

6 DevOps: Infraestrutura, Automação e Entrega Contínua

O VITA adota Integração Contínua (CI) para garantir que toda alteração no código seja verificada automaticamente antes de chegar ao repositório compartilhado. A Entrega Contínua (CD), nesta fase, consiste em manter a API sempre empacotável em uma imagem Docker reprodutível.

6.1 Pipeline CI/CD

O pipeline de integração e entrega contínua é fundamental para automação:

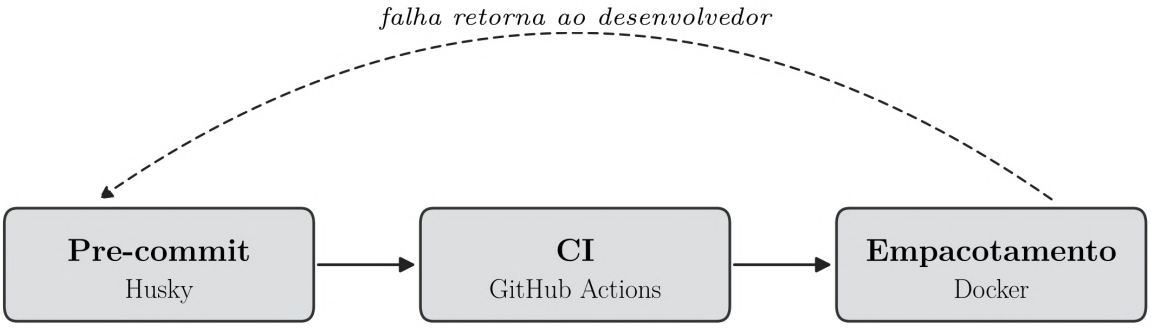


Figura 16: Pipeline CI/CD

6.2 Configuração CI/CD

Arquivo do GitHub Actions:

```
name: CI
on: push

jobs:
  validar:
    name: Lint, tipos e build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: pnpm/action-setup@v6
        with:
          version: 11
      - uses: actions/setup-node@v7
        with:
          node-version: 24
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm format:check
      - run: pnpm type-check
      - run: pnpm build
```

6.3 Infraestrutura como Código

Tabela 3: Ferramentas DevOps Utilizadas

Categoria	Ferramenta	Finalidade
Controle de Versão	Git + GitHub	Versionamento e colaboração
Gerenciamento de pacotes	pnpm 11	Instalação reprodutível via lockfile
Qualidade de código	ESLint + Prettier	Análise estática e padronização de formatação
Tipagem e build	TypeScript (tsc)	Checagem de tipos e compilação
Verificação local	Husky	Hook de pre-commit
CI	GitHub Actions	Verificação automática a cada push
Containerização	Docker	Empacotamento da API em imagem
Orquestração local	Docker Compose	Execução conjunta de API e banco

6.4 Infraestrutura do Sistema

Esta seção documenta como o sistema funciona em diferentes ambientes e plataformas.

6.4.1 Arquitetura de Rede

Tabela 4: Componentes de Infraestrutura

Componente	Descrição
Rede Local (LAN)	• Servidor local para desenvolvimento e testes • Rede interna com IP fixo (ex: 192.168.1.x) • Acesso via VPN para equipe remota • Backup local diário em servidor dedicado
Cloud (Nuvem)	• Hospedagem em AWS/Vercel/Railway/Runpod • Ambiente de produção e homologação separados
Dispositivos Móveis	• API REST para comunicação mobile • Suporte para Android • Sincronização offline com cache local
Segurança	• Firewall + WAF • Cloudflare DNS • Autenticação JWT • Monitoramento de acessos e logs de auditoria

6.4.2 Diagrama de Infraestrutura

Para ilustrar como os componentes se conectam:

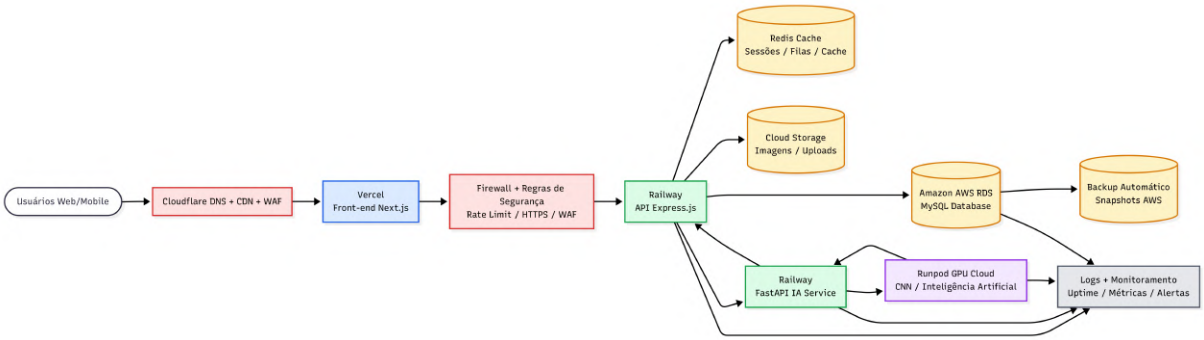


Figura 17: Diagrama de Infraestrutura do Sistema

Exemplo de descrição:

- **Usuários Web:** Acessam via navegador através de HTTPS
- **Usuários Mobile:** Conectam-se via API REST com autenticação
- **Cloudflare e Firewall:** Camadas de segurança da rede
- **Banco de Dados:** PostgreSQL hospedado na AWS RDS
- **Cache:** Redis para gerenciamento de filas
- **BullMQ:** Processamento assíncrono de tarefas
- **Storage:** Cloud Storage para imagens
- **Backup:** Automação no Snapshots AWS
- **Runpod:** Serviço de GPU em nuvem para CNN

6.5 Arquitetura de Deploy

Para documentar a arquitetura de deployment:

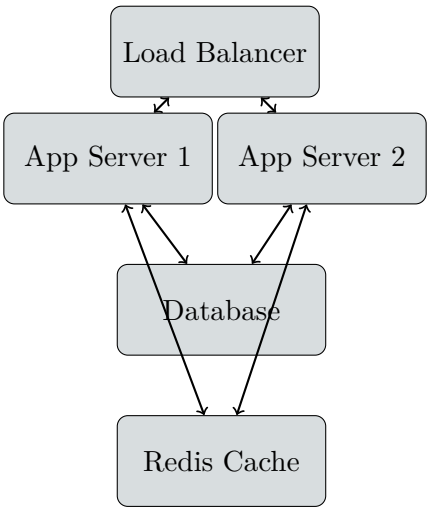


Figura 18: Arquitetura de Deployment

6.6 Métricas e SLA

Definir métricas importantes para monitoramento:

- **Disponibilidade:** 99.9% (uptime)
- **Tempo de Resposta:** $<$ 200ms (média)
- **Taxa de Erro:** $<$ 0.1%
- **Throughput:** 1000 requisições/segundo
- **Deploy Frequency:** Diário
- **Lead Time:** $<$ 1 hora
- **MTTR:** $<$ 30 minutos (Mean Time To Recovery)

7 Modelagens do Banco de Dados

Processo de planejamento e organização dos dados de um sistema, definindo como serão armazenados, relacionados e acessados. Seu objetivo é garantir eficiência, integridade e facilidade de manutenção das informações.

7.1 Modelagem conceitual

Etapa inicial da modelagem de dados que representa, de forma abstrata, as entidades, atributos e relacionamentos do sistema. Normalmente é feita sem considerar tecnologias ou bancos de dados específicos.

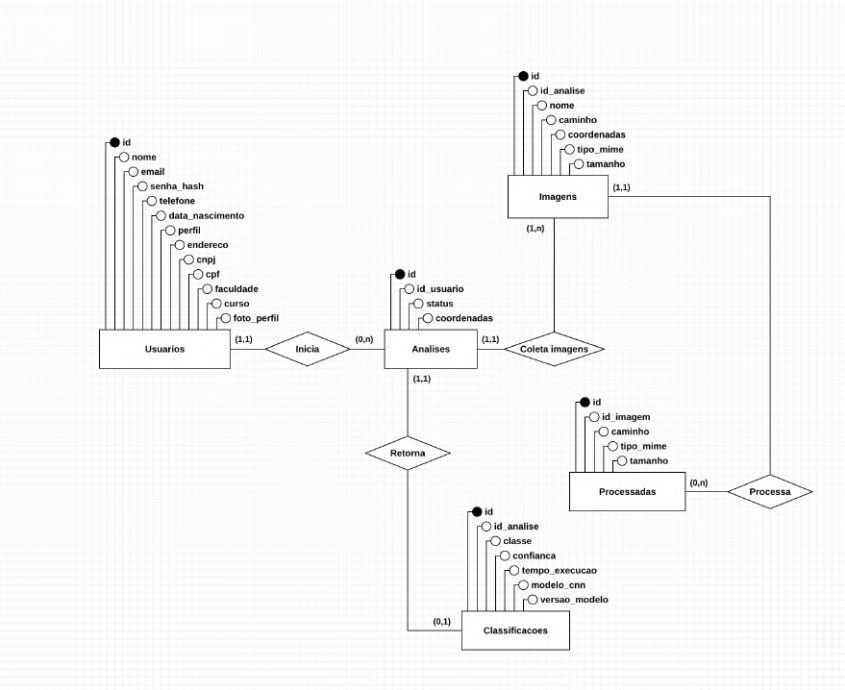


Figura 19: Modelo conceitual do Banco de Dados

7.2 Modelagem lógica

Fase que transforma o modelo conceitual em uma estrutura mais detalhada, definindo tabelas, chaves e relacionamentos conforme as regras do banco de dados. Ainda não considera detalhes físicos de implementação.

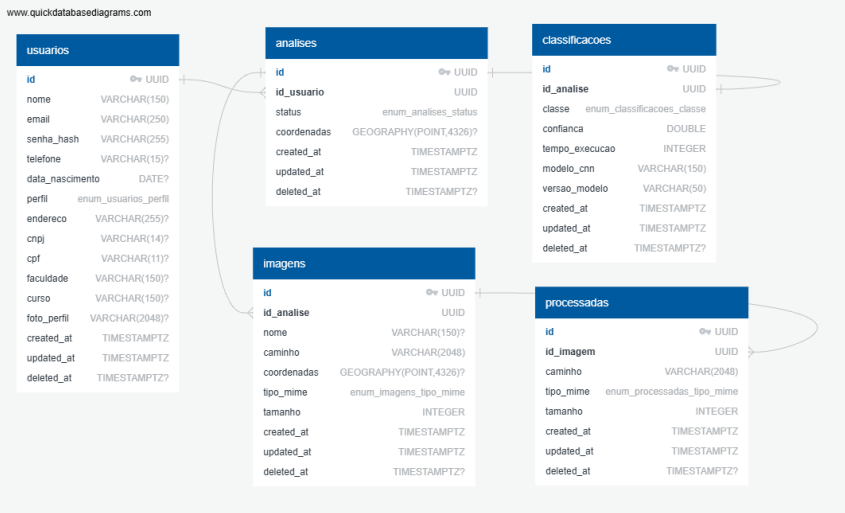


Figura 20: Modelo lógico do Banco de Dados

Conclusão

Este documento apresenta os principais artefatos necessários para a documentação completa de um projeto de software. Cada seção fornece exemplos práticos de como inserir e formatar os diferentes tipos de artefatos utilizando LaTeX.

Os artefatos documentados incluem:

- **CANVAS:** Modelo de negócio e estratégia
- **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
- **UX/UI:** Design de interface e experiência do usuário
- **Website:** Estrutura e especificações técnicas web
- **Diagramas UML:** Modelagem visual do sistema
- **DevOps:** Infraestrutura, automação e práticas de entrega contínua

---

Para incluir imagens personalizadas, substitua os caminhos de exemplo (imgs/img_001.jpg) pelos caminhos reais dos seus arquivos. Utilize ferramentas especializadas como Draw.io, Figma, PlantUML e outras mencionadas ao longo do documento para criar artefatos de qualidade profissional.

Referências

KUMAR, S.; CHEN, L. Efficient algorithms for differential equation systems. In: *Proceedings of the International Conference on Computational Science*. Prague, Czech Republic: [s.n.], 2024. p. 112–125.

OLIVEIRA, P.; COSTA, M. Machine learning applications in dynamic systems. *IEEE Transactions on Systems, Man, and Cybernetics*, IEEE, v. 53, n. 8, p. 4567–4580, 2023.

SILVA, J.; ALMEIDA, R.; RODRIGUES, T. Computational modeling of nonlinear phenomena. *Applied Mathematics and Computation*, Elsevier, v. 412, p. 126589, 2025.